

CUDA Parallel Programming Project Final Report

Torge Schwark

May 2, 2026

Contents

1	Problem Definition and Sequential Algorithm	2
1.1	Problem Description	2
1.2	Sequential Algorithm	2
1.3	Iterative Cooley–Tukey FFT and Bit-Reversal	3
2	GPU Approach	5
2.1	Parallel FFT Implementation on the GPU	5
2.1.1	Adapting the Algorithm for Parallel Execution	5
2.1.2	Parallel FFT Stage Computation	5
2.1.3	CPU vs. GPU Responsibilities	6
2.1.4	Performance Considerations	6
3	Results	8
3.0.1	Comparison of Parallel FFT Implementations	8
3.0.2	Summary and Discussion	10
3.0.3	Influence of Block Size on Performance	11

Chapter 1

Problem Definition and Sequential Algorithm

1.1 Problem Description

This Project discusses how to implement the Fast Fourier Transformation(FFT) efficiently on GPU. Before looking into the implementation as a sequential or parallel program it is important to understand the mathematical basis of the FFT. The Discrete Fourier Transformation DFT is defined by the Following formula.

$$X_k = \sum_{n=0}^{N-1} x_n e^{-i\frac{2\pi}{N}kn}, \quad k = 0, 1, \dots, N-1 \quad (1.1)$$

where:

- x_n is the input signal in the time domain,
- X_k is the resulting frequency domain representation,
- N is the total number of samples,
- n is the time-domain index,
- k is the frequency-domain index,
- $e^{-i\frac{2\pi}{N}kn}$ is the complex exponential basis function.

X_k is a complex number representing the amplitude and phase of the frequency component k .

1.2 Sequential Algorithm

One popular algorithm to compute the Fast Fourier Transform (FFT) is the Cooley–Tukey algorithm. It can be implemented in an iterative form, which is commonly used on CPUs. On GPUs, the algorithm is typically parallelized in each of the $\log(n)$ stages.

The idea of the Cooley-Tukey algorithm is to use a divide and concur approach, to solve the FFT for two halves first: Inputs with even idices and inputs with odd indices. These can than be combined using the so called twiddle factors to get the result of the complete FFT.

$$X_k = E_k + W_N^k O_k \quad (1.2)$$

$$X_{k+\frac{N}{2}} = E_k - W_N^k O_k \quad (1.3)$$

where:

$$E_k = \sum_{n=0}^{\frac{N}{2}-1} x_{2n} e^{-i \frac{2\pi}{N/2} kn} \quad (1.4)$$

$$E_k = \sum_{n=0}^{\frac{N}{2}-1} x_{2n} e^{-i \frac{2\pi}{N/2} kn} \quad (1.5)$$

and

$$W_N^k = e^{-i \frac{2\pi}{N} k} \quad (1.6)$$

1.3 Iterative Cooley–Tukey FFT and Bit-Reversal

To implement the Cooley–Tukey algorithm efficiently on a CPU, the recursive formulation is typically transformed into an iterative, in-place algorithm. This requires a preprocessing step known as *bit-reversal permutation*, followed by a sequence of $\log_2(N)$ iterative stages.

Bit-Reversal Permutation

The divide-and-conquer structure of the Cooley–Tukey algorithm assumes that the input is recursively split into even and odd indexed elements. In the iterative formulation, this ordering is achieved explicitly by permuting the input array according to reversed binary indices.

For an input of size $N = 2^m$, each index $i \in [0, N - 1]$ can be represented as an m -bit binary number:

$$i = (b_{m-1}b_{m-2} \dots b_0)_2.$$

The bit-reversed index j is obtained by reversing the order of the bits:

$$j = (b_0b_1 \dots b_{m-1})_2.$$

The input array is then reordered such that each element at index i is swapped with the element at index j . This ensures that elements which would be combined in the recursive formulation are already placed at the correct positions for the iterative computation.

This permutation allows the FFT to be computed in-place without additional memory and eliminates the need for recursive function calls.

Iterative FFT Stages

After the bit-reversal step, the FFT is computed in $\log_2(N)$ stages. Each stage combines smaller discrete Fourier transforms into larger ones using the butterfly operation.

At stage s , subproblems of size $m = 2^s$ are processed. For each block of size m , pairs of elements are combined:

$$u = x[k + j], \quad t = W_m^j \cdot x[k + j + m/2],$$

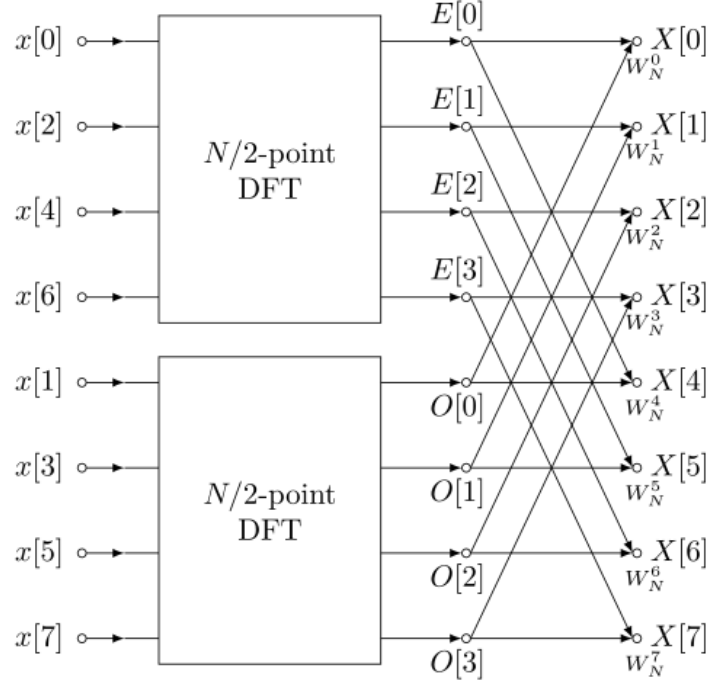


Figure 1.1: Bit-reversal permutation used in the Cooley–Tukey FFT algorithm [2].

where the twiddle factor is defined as:

$$W_m^j = e^{-i\frac{2\pi}{m}j}.$$

The butterfly operation then computes:

$$x[k + j] = u + t,$$

$$x[k + j + m/2] = u - t.$$

Here, k denotes the starting index of the current block, and j runs from 0 to $m/2 - 1$.

Each stage doubles the size of the transformed subsequences, starting from size 2 up to N . After $\log_2(N)$ stages, the full discrete Fourier transform is obtained.

Summary

The iterative Cooley–Tukey FFT consists of:

- A bit-reversal permutation to reorder the input data,
- $\log_2(N)$ stages of butterfly operations,
- The use of twiddle factors to correctly combine partial results.

This approach reduces the computational complexity from $\mathcal{O}(N^2)$ to $\mathcal{O}(N \log N)$ while allowing efficient in-place computation.

Chapter 2

GPU Approach

2.1 Parallel FFT Implementation on the GPU

2.1.1 Adapting the Algorithm for Parallel Execution

To transform the iterative Cooley–Tukey FFT into a parallel GPU implementation, the core structure of the algorithm must be adapted to expose parallelism.

The most important observation is that all butterfly operations within a single stage are independent. Therefore, instead of computing them sequentially in nested loops, each butterfly can be assigned to a separate GPU thread.

The outer loop over the FFT stages (i.e., $m = 2, 4, \dots, N$) remains sequential and is executed on the host. For each stage, a kernel is launched that processes all $\frac{N}{2}$ butterfly operations in parallel.

Bit-Reversal on the GPU The bit-reversal permutation is also parallelized. In contrast to the CPU implementation, where indices are incrementally updated, each GPU thread independently computes the bit-reversed index of its assigned element.

Given an index i , its bit-reversed counterpart j is computed by iteratively reversing its binary representation:

$$j = \sum_{k=0}^{\log_2(N)-1} \left(\frac{i}{2^k} \bmod 2 \right) 2^{\log_2(N)-1-k}.$$

Each thread performs this computation independently and swaps elements if $i < j$. This approach avoids sequential dependencies and allows the permutation to be performed in parallel. However, compared to the CPU version, this method may involve slightly more arithmetic per thread.

2.1.2 Parallel FFT Stage Computation

For each FFT stage of size m , a GPU kernel is launched where each thread computes exactly one butterfly operation.

Given a thread index `tid`, the corresponding butterfly is determined by:

- The group index: `group = ⌊tid/(m/2)⌋`
- The local index within the group: `j = tid mod (m/2)`

- The starting index of the block: $k = \text{group} \cdot m$

Each thread then computes:

$$u = x[k + j], \quad t = W_m^j \cdot x[k + j + m/2],$$

followed by the butterfly update:

$$x[k + j] = u + t, \quad x[k + j + m/2] = u - t.$$

The twiddle factor is computed per thread as:

$$W_m^j = e^{-i\frac{2\pi}{m}j}.$$

2.1.3 CPU vs. GPU Responsibilities

The implementation is split between host (CPU) and device (GPU):

- **CPU (Host):**
 - Memory allocation and data transfer between host and device
 - Launching kernels for bit-reversal and each FFT stage
 - Iterating over the $\log_2(N)$ stages
- **GPU (Device):**
 - Parallel execution of bit-reversal permutation
 - Parallel computation of butterfly operations within each stage

This division allows the GPU to handle the computationally intensive parts, while the CPU manages control flow.

2.1.4 Performance Considerations

The performance of the parallel FFT implementation depends on several key factors:

Parallelism Each stage executes $\frac{N}{2}$ butterfly operations in parallel. For large N , this allows high utilization of GPU cores. However, smaller FFT sizes may not fully utilize the hardware.

Memory Access Patterns Global memory access is a critical bottleneck. The bit-reversal permutation introduces non-coalesced memory accesses, which can reduce performance. Similarly, butterfly operations access elements with a stride that increases with m .

Kernel Launch Overhead Each FFT stage requires a separate kernel launch. Since there are $\log_2(N)$ stages, this overhead can become significant for small problem sizes.

Twiddle Factor Computation Computing trigonometric functions (`sinf`, `cosf`) inside each thread is expensive. This can be optimized by:

- Precomputing twiddle factors
- Using lookup tables
- Exploiting shared memory

Memory Bandwidth The FFT is typically memory-bound rather than compute-bound. Efficient use of memory (e.g., coalesced access, shared memory) is essential for high performance.

Occupancy and Thread Configuration Choosing appropriate block sizes (e.g., 256 or 512 threads per block) affects GPU occupancy and overall throughput.

Summary

Compared to the CPU implementation, the GPU version:

- Parallelizes butterfly operations within each stage
- Computes bit-reversal independently per thread
- Uses the CPU for control flow and the GPU for computation

This approach enables significant speedups for large input sizes, while introducing new challenges related to memory access and kernel management.

Chapter 3

Results

3.0.1 Comparison of Parallel FFT Implementations

The three implementations differ mainly in how bit-reversal and twiddle factors are computed, as well as in indexing efficiency and memory usage.

Bit-Reversal

- **Approach 1 and 3:** Use a dedicated `reverse.bits` function with a precomputed $\log_2(N)$. This avoids loops and results in faster, more predictable execution.
- **Approach 2:** Computes bit-reversal iteratively per thread using a loop. This is more flexible but introduces additional arithmetic and control flow overhead.

Twiddle Factor Computation

- **Approach 1:** Computes twiddle factors using integer exponentiation (`cpow_int`) from a base twiddle. This avoids trigonometric functions but introduces multiple complex multiplications per thread.
- **Approach 2:** Computes twiddle factors directly using `sinf/cosf` via `__sincosf`. This is simple but computationally expensive due to transcendental functions.
- **Approach 3:** Uses precomputed twiddle factors stored in global memory. Threads only perform a memory lookup, avoiding both trigonometric and iterative computations.

Memory Access and Optimization

- **Approach 1:** Minimal memory overhead, but more arithmetic per thread.
- **Approach 2:** No additional memory usage, but high compute cost due to trigonometric evaluations.
- **Approach 3:** Uses additional global memory for twiddle storage and benefits from cached reads (`__ldg`). This significantly reduces arithmetic cost and improves performance.

Performance Characteristics

- **Approach 1:** Balanced approach; avoids trig functions but still compute-heavy due to exponentiation.
- **Approach 2:** Simplest implementation but typically the slowest due to expensive trigonometric operations.
- **Approach 3:** Most optimized; minimizes computation by trading it for memory access.

In the following, we refer to the first approach as **Base**, since it uses a base twiddle factor together with an exponentiation function to compute the required values for the butterfly operations. The second approach is referred to as **Normal**, as each thread computes the twiddle factors directly according to the mathematical definition. The third approach is referred to as **Pre**, since the twiddle factors are precomputed and stored in memory.

Additionally, the term **Sequential** refers to the non-parallel baseline implementation executed on the GPU without optimization.

The results are measured in comparison to the CUDA FFT library (cuFFT) [1].

Table 3.1: Comparison Block Size 1024

N	CPU	Sequential	Base	Pre	Normal
16384	0.07	3.89	2.00	2.60	3.44
32768	0.08	7.43	1.79	2.45	3.89
65536	0.12	11.19	1.57	2.07	2.57
131072	0.15	21.80	1.47	2.55	3.22
262144	0.20	33.07	1.09	1.67	3.46
524288	0.20	72.96	1.65	2.46	5.11
1048576	0.26	145.84	3.37	4.43	7.48
2097152	0.38	232.37	4.56	6.10	10.09
4194304	0.63	319.19	5.51	23.57	13.06
8388608	1.58	292.04	5.90	22.87	9.92

Table 3.2: Comparison Block Size 512

N	CPU	Sequential	Base	Pre	Normal
16384	0.06	4.58	2.15	2.45	2.80
32768	0.08	7.70	1.48	2.27	2.96
65536	0.09	14.32	1.97	3.55	2.92
131072	0.14	20.73	1.23	3.31	3.32
262144	0.18	35.84	1.16	1.78	3.31
524288	0.19	76.62	2.01	2.45	4.76
1048576	0.27	115.22	3.09	4.36	6.60
2097152	0.33	201.26	5.16	6.74	10.53
4194304	0.53	324.11	6.94	8.75	14.55
8388608	1.05	372.37	8.86	32.14	12.39

Table 3.3: Comparison Block Size 256

N	CPU	Sequential	Base	Pre	Normal
16384	0.06	4.83	2.32	2.72	3.21
32768	0.07	9.77	1.94	2.27	3.05
65536	0.12	11.36	1.48	2.38	2.98
131072	0.19	15.40	1.20	2.12	1.98
262144	0.19	35.85	1.30	1.57	2.88
524288	0.20	73.38	1.86	2.51	4.50
1048576	0.26	120.45	3.28	4.66	6.77
2097152	0.32	237.16	5.33	7.06	10.68
4194304	0.61	322.83	5.60	26.73	14.19
8388608	1.09	349.88	8.55	46.21	10.62

Table 3.4: Comparison Block Size 128

N	CPU	Sequential	Base	Pre	Normal
16384	0.09	3.40	1.34	3.08	2.65
32768	0.07	8.76	2.07	2.65	3.11
65536	0.10	13.57	1.68	2.54	3.37
131072	0.14	20.12	1.51	2.79	2.67
262144	0.21	32.30	1.42	1.60	2.88
524288	0.22	68.47	1.60	2.26	4.25
1048576	0.27	118.74	3.22	4.34	6.53
2097152	0.34	212.79	5.10	6.63	10.18
4194304	0.65	263.78	5.29	8.05	17.35
8388608	1.59	245.20	5.85	22.88	9.43

3.0.2 Summary and Discussion

Figure 3.2 shows a comparison of the different FFT implementations. It can be observed that the **Base** approach consistently achieves the best performance among the custom GPU implementations. In particular, for larger input sizes, the Base twiddle algorithm clearly outperforms the **Normal** and **Pre** variants.

The main reason for this behavior lies in the computation of the twiddle factors. The Base approach avoids expensive trigonometric function evaluations by computing powers of a base twiddle factor using repeated complex multiplications. This reduces the computational cost compared to the Normal approach, where each thread evaluates sine and cosine functions. At the same time, it avoids the additional memory overhead and bandwidth pressure introduced by the Pre approach, which relies on loading precomputed twiddle factors from global memory. As a result, the Base method achieves a good balance between arithmetic intensity and memory access efficiency.

However, despite being the fastest custom implementation, the Base approach is still slower than the highly optimized cuFFT library. This is expected, as cuFFT is specifically designed for high-performance FFT computations on GPUs. It employs a number of advanced optimization techniques that are not present in the custom implementations, such as optimized memory layouts, kernel fusion, reduced global memory traffic, and architecture-specific tuning. Furthermore, cuFFT makes extensive use of shared memory, instruction-level optimizations, and minimizes synchronization overhead across stages.

Table 3.5: Comparison Block Size 64

N	CPU	Sequential	Base	Pre	Normal
16384	0.07	4.21	1.65	2.19	2.75
32768	0.07	8.75	2.06	2.61	3.28
65536	0.10	14.29	1.50	2.57	2.64
131072	0.14	21.65	1.12	2.73	2.71
262144	0.19	34.80	1.01	2.00	3.19
524288	0.23	65.37	1.44	2.17	3.96
1048576	0.26	126.55	3.37	4.44	6.79
2097152	0.33	213.64	5.22	6.96	10.46
4194304	0.69	257.60	4.97	7.05	15.87
8388608	1.06	364.35	8.72	31.40	15.44

Table 3.6: Comparison Block Size 32

N	CPU	Sequential	Base	Pre	Normal
16384	0.06	4.61	2.07	2.83	2.79
32768	0.08	7.92	1.94	2.43	2.55
65536	0.10	14.60	1.74	2.88	2.61
131072	0.15	19.66	1.22	2.47	2.52
262144	0.18	36.38	1.21	1.67	3.31
524288	0.20	73.33	1.52	2.35	4.70
1048576	0.25	175.01	3.59	4.59	6.90
2097152	0.37	221.18	4.61	6.08	9.29
4194304	0.81	253.42	4.24	31.68	9.16
8388608	1.05	419.70	8.89	29.90	12.28

In summary, the Base twiddle implementation provides the best performance among the evaluated approaches due to its efficient trade-off between computation and memory usage. Nevertheless, vendor-optimized libraries such as cuFFT remain superior due to their highly specialized and hardware-aware optimizations.

3.0.3 Influence of Block Size on Performance

Figure ?? shows the performance of the different GPU implementations for varying block sizes at a fixed input size of $N = 4194304$.

The **Base** approach achieves the best performance among the custom implementations, with its optimal configuration at a block size of 64. For smaller block sizes, the performance is limited due to insufficient parallelism, while larger block sizes lead to reduced efficiency caused by resource constraints such as register usage and limited occupancy.

The **Pre** approach performs consistently worse than the Base implementation due to additional global memory accesses required for loading precomputed twiddle factors. The **Normal** approach shows the weakest performance, as it requires expensive trigonometric computations within each thread.

Although the Base implementation is the fastest among the custom approaches, all implementations remain slower than cuFFT. This is due to the highly optimized nature of the library, which leverages hardware-specific optimizations and more efficient memory usage.

Table 3.7: Comparison Block Size 16					
N	CPU	Sequential	Base	Pre	Normal
16384	0.06	4.55	2.71	3.15	3.97
32768	0.07	8.94	3.01	3.27	3.54
65536	0.09	14.52	2.24	3.36	3.74
131072	0.14	20.67	1.51	2.93	3.60
262144	0.18	36.35	1.52	2.29	4.77
524288	0.20	72.63	2.38	3.12	7.72
1048576	0.25	124.05	3.55	4.77	12.17
2097152	0.33	231.21	5.38	7.20	18.56
4194304	0.61	316.44	5.65	20.00	23.85
8388608	1.07	229.26	7.99	31.43	20.79

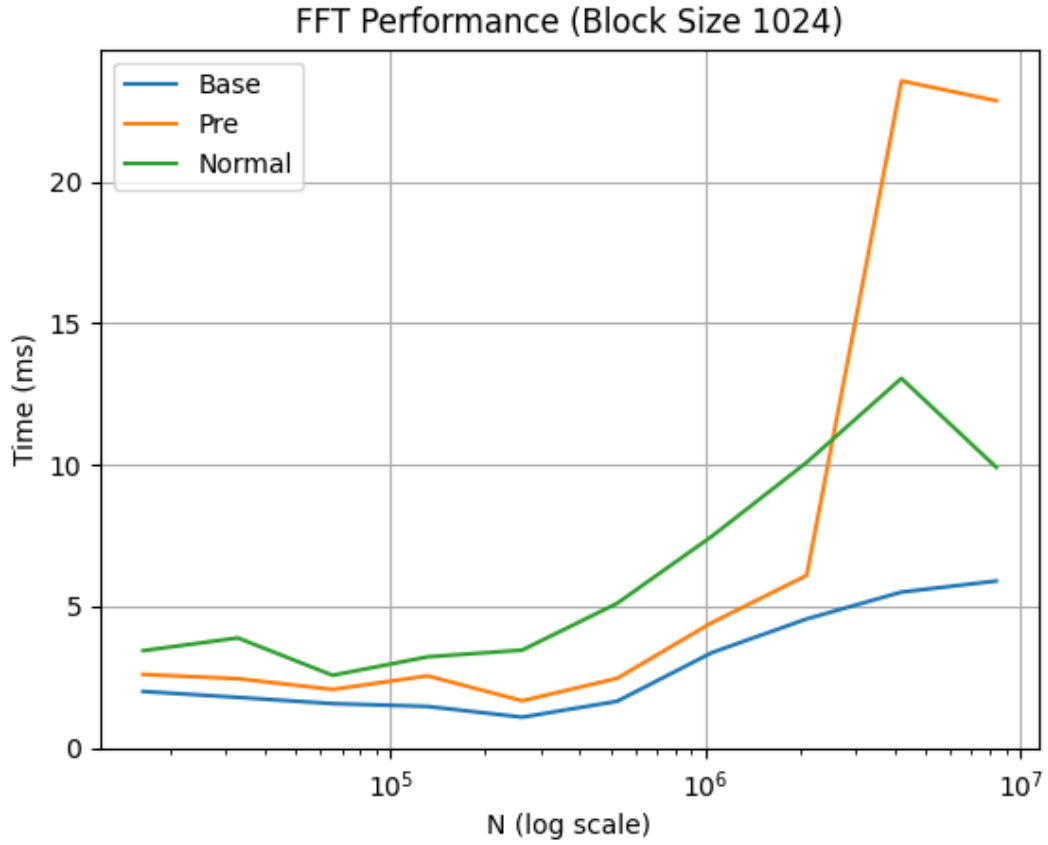


Figure 3.1: Comparison different approaches

Overall, a medium block size provides the best balance between parallelism and hardware utilization.

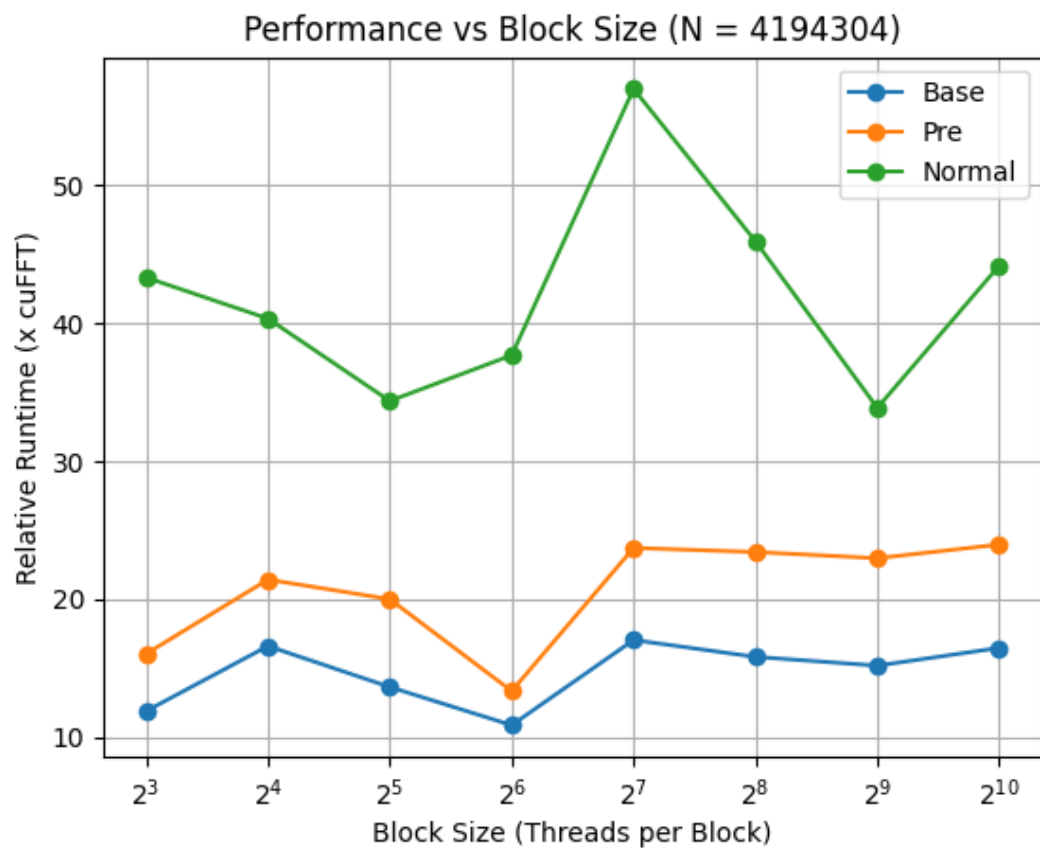


Figure 3.2: Comparison different block sizes (20 Measurements)

Bibliography

- [1] NVIDIA Corporation, *cuFFT Library User Guide*, <https://docs.nvidia.com/cuda/cufft/index.html>, accessed May 2, 2026.
- [2] Wikipedia contributors, *Cooley–Tukey FFT algorithm*, https://en.wikipedia.org/wiki/Cooley%E2%80%93Tukey_FFT_algorithm, accessed May 2, 2026.